

# Proyecto Final

## Diseño e Implementación de Arquitectura NoSQL Geoespacial para Análisis de Potencial Solar en Territorios PDET

Daniel González, Eduardo De Brigard, Juan Pablo Hernández

20 de mayo de 2026

### 1. Introducción

En el marco de la transición energética impulsada por la Unidad de Planeación Minero Energética (UPME), surge la necesidad de identificar zonas estratégicas para el despliegue de infraestructura de energía renovable en Colombia. Este proyecto se enfoca específicamente en evaluar el potencial de generación solar en los techos de edificaciones dentro de los territorios priorizados por los Programas de Desarrollo con Enfoque Territorial (PDET), buscando promover la equidad territorial y el desarrollo en regiones históricamente vulnerables. El principal desafío técnico radica en el procesamiento masivo de datos. Para estimar el área disponible para paneles solares, es necesario integrar y comparar datasets globales que contienen miles de millones de registros de huellas de edificios (como Google Open Buildings y Microsoft Building Footprints), junto con la delimitación administrativa del DANE. Dada la escala de la información y la naturaleza geográfica de los datos, este documento presenta una propuesta basada en tecnologías NoSQL, específicamente utilizando MongoDB. El uso de este paradigma permite una gestión flexible de esquemas y, sobre todo, una ejecución eficiente de operaciones espaciales complejas, fundamentales para cruzar los límites municipales con las estructuras detectadas por satélite.

### 2. Diseño del esquema de base de datos NoSQL y plan de implementación

#### 2.1. Plan de Implementación

##### 1. Preparación de la Infraestructura NoSQL

- Despliegue de MongoDB: Configuración de una instancia de base de datos orientada a documentos para el almacenamiento masivo de datos vectoriales.
- Indexación Espacial: Implementación de índices de tipo `2dsphere` para optimizar las consultas de proximidad y contención geográfica.

##### 2. Adquisición e Integración de Datos Territoriales

- Filtro PDET: Procesamiento del Marco Geoestadístico Nacional (MGN) del DANE para extraer y limpiar los límites de los municipios priorizados como territorios PDET.
- Ingesta de Huellas de Edificios: Carga y normalización de al menos dos de los datasets globales (Microsoft Building Footprints y Google Open Buildings) en la base de datos NoSQL.
- Verificación de Integridad: Validación de formatos estructurales, topología de geometrías

y consistencia de códigos DIVIPOLA para asegurar la integridad referencial y la precisión espacial de los datos territoriales integrados en el esquema NoSQL.

### 3. Procesamiento y Análisis Geoespacial

- Intersección Espacial: Ejecución de operaciones de filtrado para identificar exclusivamente las estructuras ubicadas dentro de los polígonos municipales PDET.
- Cálculo de Áreas: Estimación de la superficie de los techos y conteo total de edificaciones por cada jurisdicción seleccionada.

### 4. Evaluación y Comparativa de Fuentes

- Auditoría de Datos (EDA): Realización de un análisis exploratorio para comparar la precisión y el volumen de detecciones entre los datasets seleccionados.
- Análisis de Discrepancias: Definición de métricas de validación para identificar variaciones en la detección de estructuras y determinar la fuente con mayor confiabilidad para el cálculo de áreas solares en los municipios seleccionados.

### 5. Consolidación de Resultados

- Generación de Reportes: Creación de visualizaciones cartográficas y tablas de resumen que identifiquen los municipios con mayor potencial para granjas solares.
- Documentación de Recomendaciones: Redacción de hallazgos técnicos alineados con los objetivos de transición energética y equidad territorial de la UPME.

## 2.2. Diseño del Modelo de Datos

### 2.2.1. A. Representación del Modelo (Estructura de Documentos)

A continuación, se presentan las colecciones principales definidas para el almacenamiento y procesamiento de información geoespacial dentro del entorno NoSQL propuesto.

#### **Colección: territorios\_pdet**

- `_id`: Identificador único (ObjectId)
- `codigo_mgn`: Código municipal del DANE
- `nombre_municipio`: Nombre oficial
- `geometria_pdet`: Objeto GeoJSON (MultiPolygon) que define los límites del municipio
- `datos_pdet`: Información sobre el programa de desarrollo

#### **Colección: huellas\_edificios**

- `_id`: Identificador único (ObjectId)
- `fuelle`: Origen del dato (Microsoft o Google)
- `geometria_edificio`: Objeto GeoJSON (Polygon) con la huella del edificio
- `area_estimada_m2`: Valor numérico calculado para el potencial solar
- `metadata`: Confianza de detección y fecha de captura

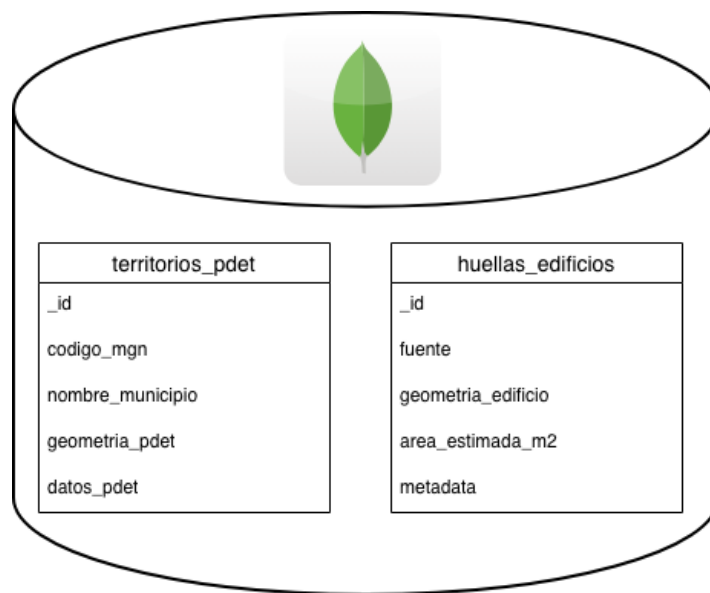


Figura 1: Esquema de base de datos

### 2.2.2. B. Instrumento de Modelado: Relación Espacial

En este modelo, la relación no se define mediante claves foráneas como en bases de datos relacionales, sino mediante una lógica de contención espacial:

- **Vínculo lógico:** Un documento en `huellas_edificios` pertenece a un territorio en `territorios_pdet` si su `geometria_edificio` está contenida dentro de `geometria_pdet`.
- **Mecanismo de consulta:** Se utilizará el operador `$geoWithin` de MongoDB para realizar el cruce de datos en tiempo de ejecución, evitando redundancia innecesaria.

## 2.3. Esquema de Datos y Justificación

Para cumplir con el requerimiento de una solución escalable y eficiente en operaciones espaciales, se ha diseñado un esquema basado en dos colecciones principales dentro de MongoDB.

### 2.3.1. Descripción de las Colecciones

#### Colección: `pdet_territories`

- **Propósito:** Almacenar los límites administrativos de los municipios designados como PDET, derivados del Marco Geoestadístico Nacional (MGN) del DANE.
- **Contenido:** Documentos que contienen el código municipal, nombre oficial y un campo `geometry` de tipo MultiPolygon bajo el estándar GeoJSON.
- **Índice:** Se implementará un índice `2dsphere` sobre el campo de geometría para permitir consultas de contención rápidas.

#### Colección: `building_footprints`

- **Propósito:** Consolidar las detecciones de edificios de los datasets de Google Open Buildings y Microsoft Building Footprints para su comparación.
- **Contenido:** Documentos con la huella del edificio (Polygon), el área calculada, la fuente del

dato y metadatos de confianza.

- **Optimización:** Se incluirá un campo con el código municipal (obtenido tras el procesamiento inicial) para facilitar agregaciones estadísticas sin necesidad de realizar cruces espaciales en cada consulta.

### 2.3.2. Justificación Técnica

La elección de este esquema y del motor NoSQL (MongoDB) se fundamenta en los siguientes pilares:

- **Soporte nativo geoespacial:** A diferencia de las bases de datos relacionales tradicionales, MongoDB maneja objetos GeoJSON de forma nativa. Esto facilita la ejecución de operadores como `$geoWithin` y `$geoIntersects`, esenciales para determinar qué edificios pertenecen a cada municipio PDET.
- **Escalabilidad:** El proyecto requiere procesar millones de registros provenientes de datasets globales de huellas de edificios. MongoDB permite manejar grandes volúmenes de información geoespacial de forma distribuida y eficiente.
- **Flexibilidad de esquema:** Debido a que los datasets utilizados provienen de múltiples fuentes (Google y Microsoft), existen diferencias estructurales y variaciones en atributos. El modelo orientado a documentos facilita la integración de estructuras heterogéneas sin requerir migraciones complejas.
- **Optimización de consultas espaciales:** La implementación de índices `2dsphere` permite acelerar operaciones de proximidad, intersección y contención espacial sobre polígonos territoriales y huellas de edificaciones.
- **Compatibilidad con pipelines geoespaciales:** El uso de GeoJSON facilita la interoperabilidad entre herramientas como QGIS, GeoPandas y MongoDB, permitiendo mantener consistencia espacial durante todas las etapas del procesamiento.

## 2.4. Integración de Datos Territoriales y Filtro PDET

En continuidad con el plan de implementación, se llevó a cabo la integración de la información geoespacial base para disponer de una colección preparada para análisis e ingesta en el entorno NoSQL.

### 2.4.1. Metodología de procesamiento y normalización

Para seleccionar los territorios priorizados se automatizó el cruce entre el Marco Geoestadístico Nacional (MGN) 2025 del DANE y el listado oficial de municipios PDET mediante scripts reproducibles en Python utilizando GeoPandas, Pandas y Matplotlib.

El flujo implementado permitió garantizar consistencia territorial, normalización de atributos y preparación geoespacial para posteriores procesos de ingesta en MongoDB.

Las acciones principales realizadas durante el procesamiento fueron:

- Estandarización de códigos DIVIPOLA.
- Limpieza de nombres territoriales.
- Eliminación de caracteres Unicode y tildes.
- Validación de coincidencias territoriales.
- Conversión geoespacial a formato GeoJSON.

Como insumo principal del proceso se utilizó la capa administrativa nacional correspondiente al archivo:

`MGN_ADM_MPIO_GRAFICO.shp`

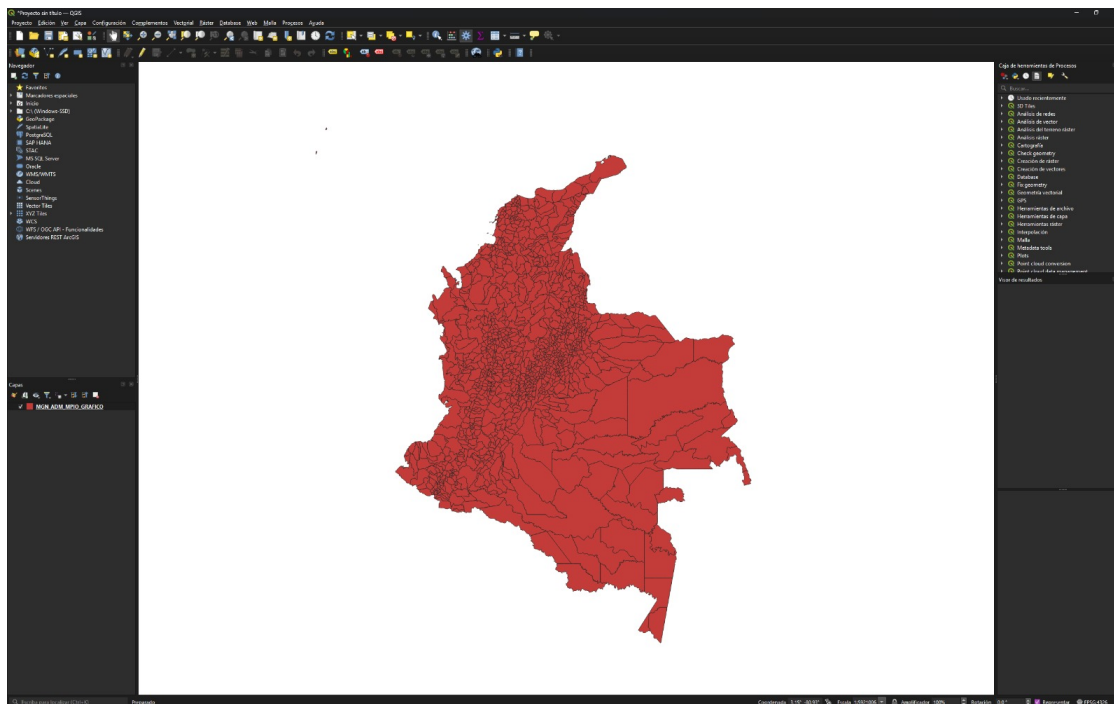


Figura 2: Visualización del shapefile administrativo `MGN_ADM_MPIO_GRAFICO.shp` cargado en QGIS. La capa representa la totalidad de municipios contenidos en el Marco Geoestadístico Nacional (MGN 2025) del DANE.

Posteriormente, el script en Python realizó el filtrado territorial correspondiente a municipios priorizados PDET y generó el archivo geoespacial:

`pdet_final.geojson`

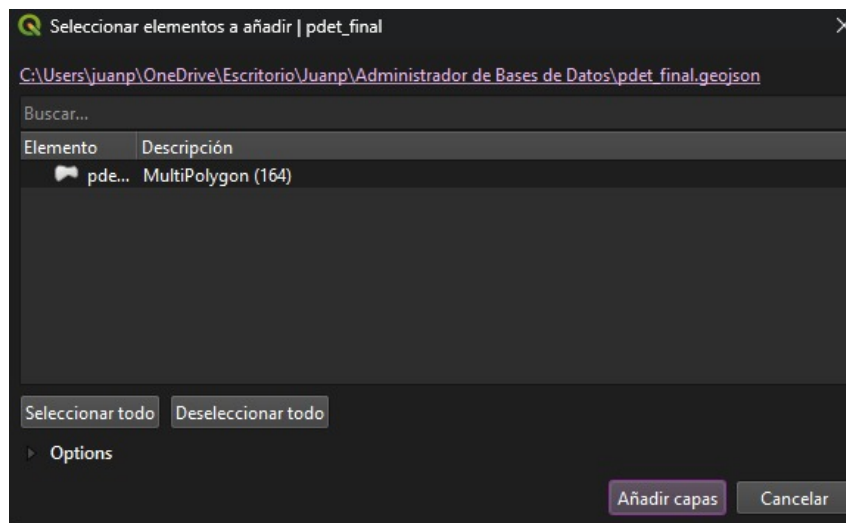


Figura 3: Visualización programática del archivo `pdet_final.geojson` generado mediante Python y GeoPandas. Cada color representa municipios PDET filtrados tras el proceso de integración territorial.

Las visualizaciones obtenidas permitieron realizar un proceso preliminar de validación espacial, verificando continuidad topológica y consistencia geométrica respecto al shapefile original del DANE.

#### 2.4.2. Validación geoespacial y control de calidad

Con el objetivo de validar la integridad geométrica del dataset resultante, se realizaron pruebas visuales mediante QGIS y validaciones programáticas utilizando Python.

Las validaciones efectuadas incluyeron:

- Superposición correcta entre el shapefile original y el GeoJSON generado.
- Validación por lotes de geometrías municipales.
- Validación individual de polígonos territoriales.
- Verificación de continuidad espacial.
- Detección de posibles desplazamientos o geometrías corruptas.

#### Superposición de capas territoriales

La superposición entre capas permitió verificar que la transformación espacial no introdujera inconsistencias territoriales ni pérdida de entidades durante la conversión.

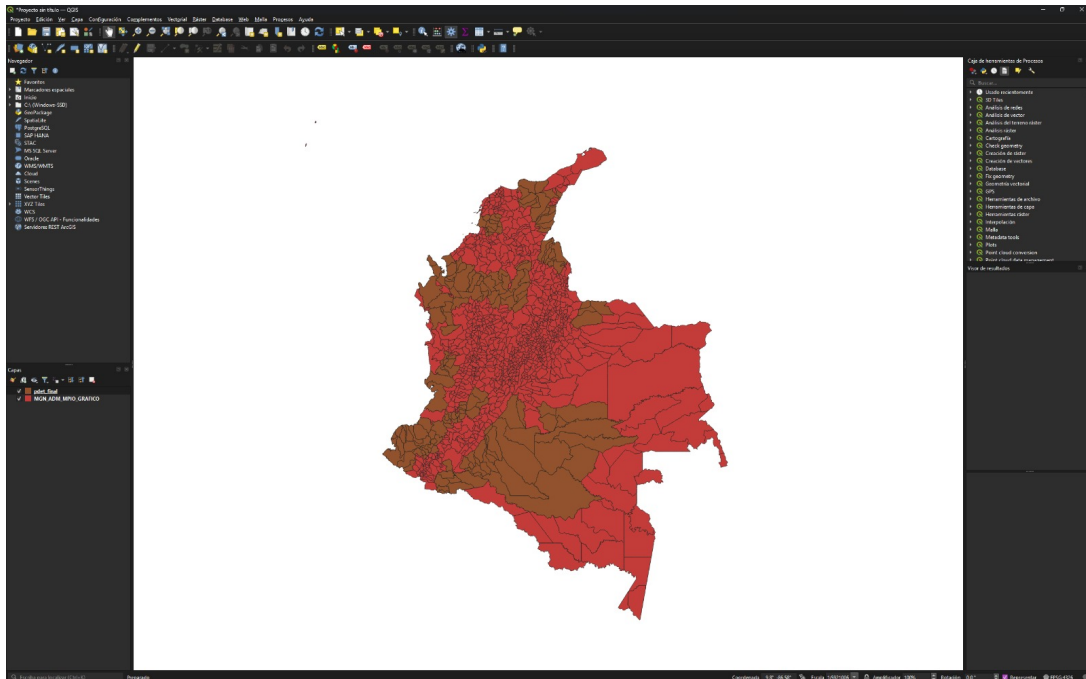


Figura 4: Superposición entre el shapefile original del DANE y el archivo `pdet_final.geojson`. La comparación permitió validar continuidad topológica y consistencia espacial entre ambas capas.

#### Validación visual mediante simbología

Adicionalmente, se realizaron pruebas manuales modificando simbologías y colores de capas dentro de QGIS con el fin de identificar posibles anomalías geométricas o desplazamientos espaciales.

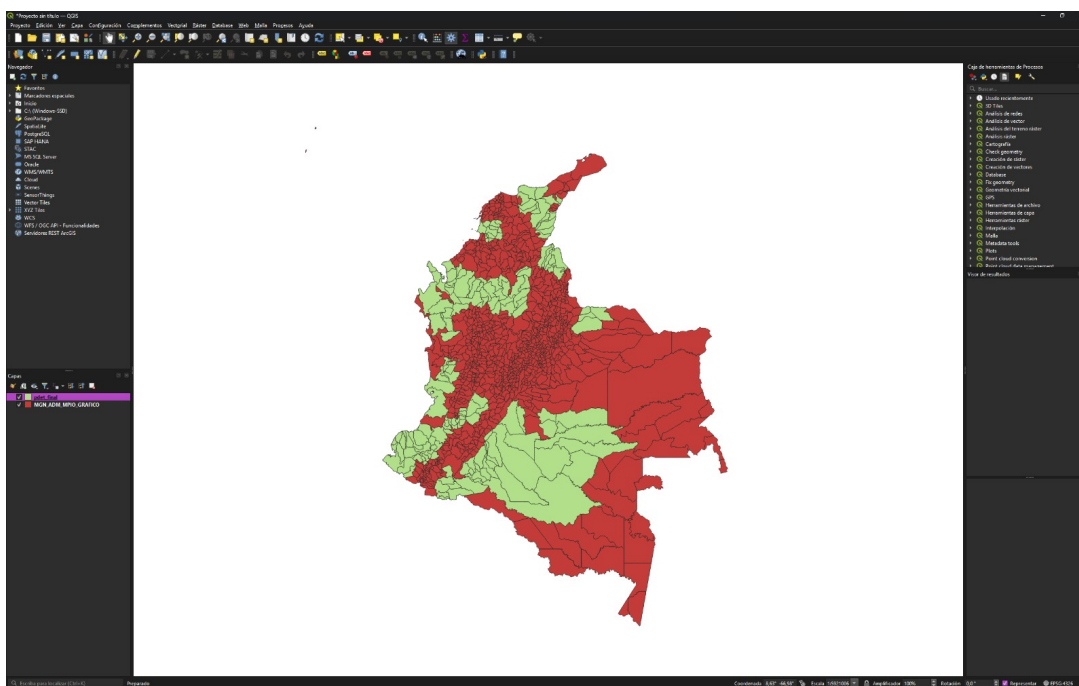


Figura 5: Validación visual mediante modificación de simbología y colores en QGIS. Este procedimiento permitió identificar posibles inconsistencias geométricas y verificar independencia topológica entre polígonos municipales.

### Validación por lotes

El proceso de control incluyó validaciones por lotes sobre múltiples geometrías municipales para verificar integridad estructural y correcta carga espacial.

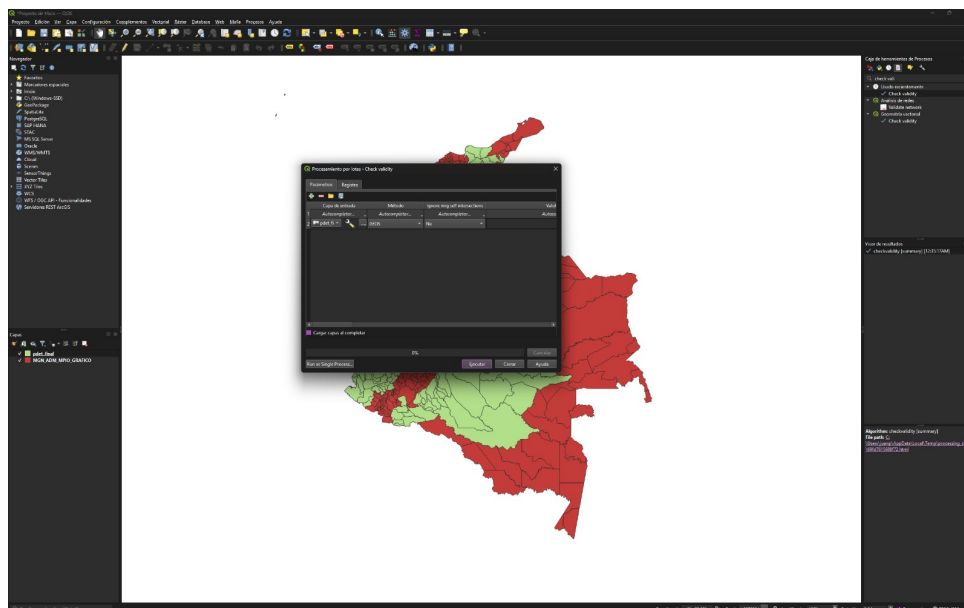


Figura 6: Proceso de validación por lotes realizado sobre múltiples geometrías municipales. La validación confirmó una carga espacial exitosa y ausencia de errores topológicos críticos.

## Validación individual de geometrías

También se realizaron inspecciones individuales sobre municipios específicos con el objetivo de corroborar la correcta representación cartográfica y creación adecuada de nuevas capas territoriales.

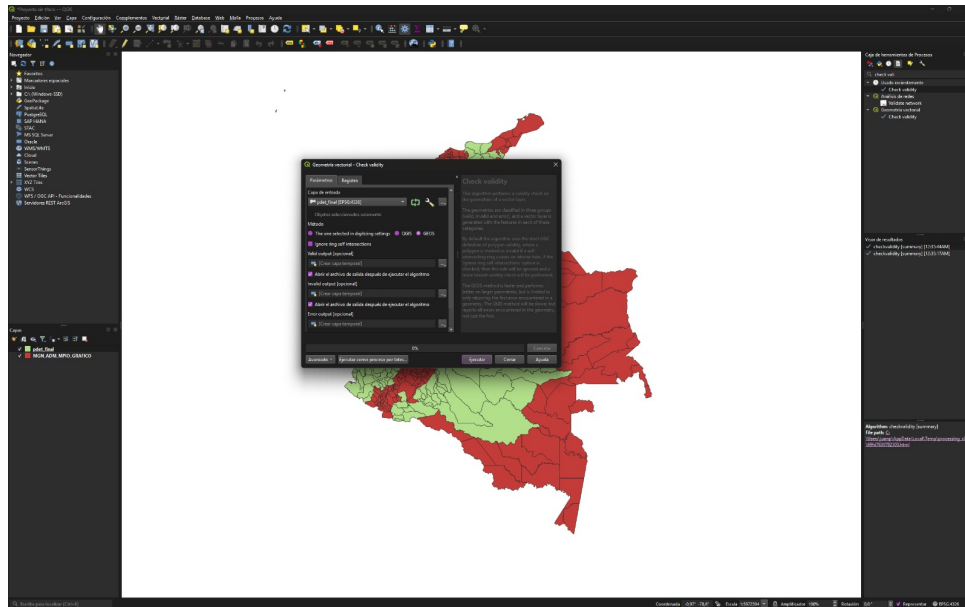


Figura 7: Validación individual de geometrías territoriales. La inspección permitió corroborar la correcta representación cartográfica de municipios específicos dentro del conjunto PDET.

## Evidencia de ejecución del pipeline

Durante la ejecución del pipeline geoespacial se obtuvieron resultados de validación en consola similares a los siguientes:

Cruce por código DANE: encontrados 170 registros.

--- RESULTADOS ---

Municipios cargados del SHP: 1123

Municipios en tu lista PDET: 170

Municipios PDET encontrados tras el cruce: 170

[OK] Archivo 'pdet\_final.geojson' creado correctamente.

Estos resultados evidencian la correcta integración territorial y la generación satisfactoria del dataset geoespacial requerido para etapas posteriores del proyecto.



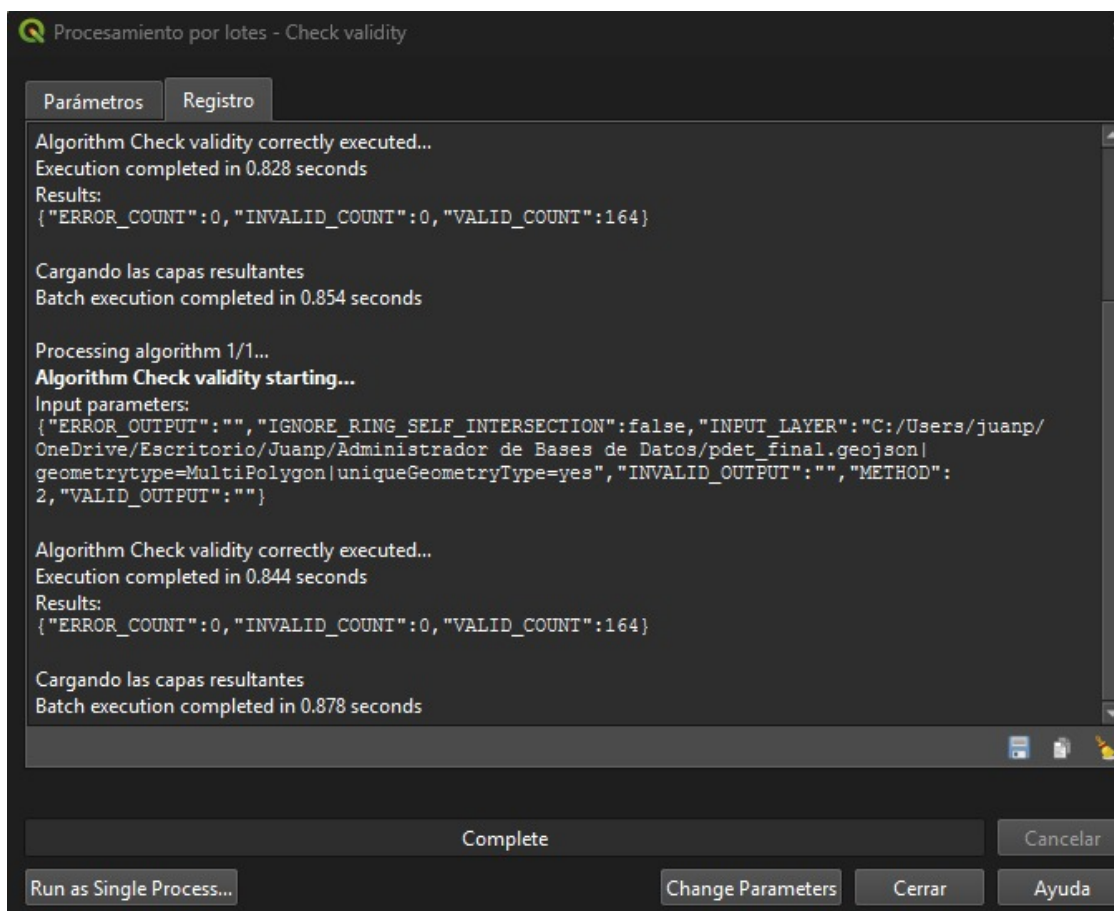


Figura 8: Salida en consola del pipeline geoespacial implementado en Python, evidenciando la carga del shapefile, integración territorial y generación del archivo GeoJSON final.

### Exportación geoespacial final

La exportación del subconjunto territorial resultante se realizó mediante la siguiente instrucción:

```
resultado_final.to_file(
    "pdet_final.geojson",
    driver="GeoJSON"
)
```

Esta transformación constituye una etapa crítica dentro del pipeline geoespacial, debido a que permite convertir estructuras territoriales tradicionales en objetos GeoJSON compatibles con MongoDB y operaciones espaciales NoSQL.

## 3. Adquisición de Datos

### 3.1. Google Open Buildings

#### 3.1.1. Descripción del dataset

Google Open Buildings es un dataset desarrollado por Google Research que contiene detecciones de edificios derivadas de imágenes satelitales de alta resolución. La versión 3 del dataset cubre Lati-

noamérica, el Caribe, África, Asia del Sur y Asia del Sureste, con un área de inferencia de 58 millones de km<sup>2</sup>.

Cuadro 1: Características del dataset Google Open Buildings

| Atributo                | Detalle                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| Versión                 | v3 (tercera versión)                                                                                            |
| Detecciones totales     | ~1.8 mil millones                                                                                               |
| Área de cobertura       | 58 millones de km <sup>2</sup>                                                                                  |
| Formato de distribución | CSV comprimido (.csv.gz) por cuadrante                                                                          |
| Atributos principales   | geometry (WKT), area_in_meters, confidence, full_plus_code                                                      |
| Licencia                | CC BY-4.0 / ODbL v1.0                                                                                           |
| URL                     | <a href="https://sites.research.google/gr/open-buildings/">https://sites.research.google/gr/open-buildings/</a> |

### 3.1.2. Proceso de descarga

Los archivos fueron descargados directamente desde el portal de Google Open Buildings, filtrando por la región correspondiente a Colombia. Cada archivo CSV comprimido contiene las huellas de edificios para un cuadrante geográfico específico, con los siguientes campos principales:

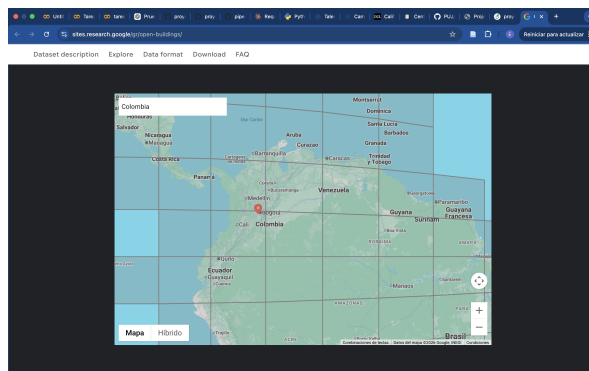


Figura 9: Google open buildings

Esto descarga unos archivos .csv.grz en los cuales estan los datos, en el script.py, se aplicó un filtro mínimo de confianza de **0.60** durante la carga, descartando detecciones con baja certeza de ser edificaciones reales.

## 3.2. Microsoft Global ML Building Footprints

### 3.2.1. Descripción del dataset

Microsoft Global ML Building Footprints es un dataset desarrollado por Microsoft Research a partir de imágenes de Bing Maps recolectadas entre 2014 y 2021, incorporando fuentes como Maxar y Airbus. Contiene más de 999 millones de detecciones de edificios a nivel mundial.

Cuadro 2: Características del dataset Microsoft Building Footprints

| Atributo                | Detalle                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Detecciones totales     | >999 millones                                                                                                                           |
| Período de imágenes     | 2014 – 2021                                                                                                                             |
| Formato de distribución | GeoJSON por cuadrante (QuadKey)                                                                                                         |
| Atributos principales   | geometry (GeoJSON Polygon), height, confidence                                                                                          |
| Licencia                | Open Data Commons Open Database License (ODbL)                                                                                          |
| URL                     | <a href="https://planetarycomputer.microsoft.com/dataset/ms-buildings">https://planetarycomputer.microsoft.com/dataset/ms-buildings</a> |

### 3.2.2. Proceso de descarga

Microsoft distribuye sus datos a través de un índice CSV que lista todos los cuadrantes disponibles por país. El proceso de descarga se basó en el snippet oficial de Microsoft, adaptado para filtrar únicamente los cuadrantes correspondientes a Colombia, este proceso de descarga se puede ver en el pruebasDosMicro.py:

```
INDEX_URL = "https://mimedbuildings.z5.web.core.windows.net/global-buildings/dataset-links.csv"
LOCATION = "Colombia"
OUTPUT_FOLDER = "microsoft_geojson"

def main():
    log.info("=== Descarga Microsoft Buildings - Colombia ===")

    os.makedirs(OUTPUT_FOLDER, exist_ok=True)

    # --- 1. Descargar índice oficial de Microsoft ---
    log.info("Descargando índice de cuadrantes...")
    dataset_links = pd.read_csv(INDEX_URL)
    colombia_links = dataset_links[dataset_links.Location == LOCATION].reset_index(drop=True)
    log.info(f"Cuadrantes de {LOCATION}: {len(colombia_links)}")

    # --- 2. Descargar cada cuadrante ---
    ok = 0
    errores = 0

    for i, row in colombia_links.iterrows():
        quadkey = row.QuadKey
        url = row.Url
        destino = os.path.join(OUTPUT_FOLDER, f"{quadkey}.geojson")

        # Si ya existe, saltar
        if os.path.exists(destino):
            log.info(f" [{i+1}/{len(colombia_links)}] Ya existe: {quadkey}.geojson")
            ok += 1
            continue

        try:
            log.info(f" [{i+1}/{len(colombia_links)}] Descargando: {quadkey} ...")

            # Igual que el snippet oficial de Microsoft
            df = pd.read_json(url, lines=True)
            df["geometry"] = df["geometry"].apply(shape)
            gdf = gpd.GeoDataFrame(df, crs=4326)
            gdf.to_file(destino, driver="GeoJSON")

            log.info(f"    -> {len(gdf),} edificios guardados en {quadkey}.geojson")
            ok += 1
        except Exception as e:
            log.error(f"Error al descargar {quadkey}: {e}")
            errores += 1
```

Figura 10: Google open buildings

Para Colombia se identificaron y descargaron **229 cuadrantes GeoJSON**, cada uno correspondiente a un QuadKey único del sistema de indexación de Bing Maps.

## 4. Integración en MongoDB e Indexación Espacial

### 4.1. Arquitectura de colecciones

Los datos fueron almacenados en tres colecciones dentro de la base de datos proyecto en MongoDB:

Cuadro 3: Colecciones en MongoDB

| Colección           | Documentos | Propósito                        |
|---------------------|------------|----------------------------------|
| territorios_pdet    | 164        | Límites municipales PDET         |
| google_buildings    | 2,617,289  | Huellas de edificios (Google)    |
| microsoft_buildings | 736,207    | Huellas de edificios (Microsoft) |

## 4.2. Esquema de documento

Cada edificio, independientemente de la fuente, sigue el esquema unificado definido desde la semana 1:

```

1 {
2   "_id":           ObjectId,
3   "fuente":        "google" | "microsoft",
4   "codigo_mgn":    "47001",           // código DANE del municipio
5   "nombre_municipio": "SANTA MARTA",
6   "geometria_edificio": {             // GeoJSON Polygon
7     "type": "Polygon",
8     "coordinates": [[[lng, lat], ...]]
9   },
10  "area_estimada_m2": 87.39,           // solo Google
11  "metadata": {
12    "confianza":      0.78,
13    "altura_m":       null,           // solo Microsoft cuando disponible
14    "fuente_imagen":  "Google Satellite Imagery",
15    "dataset_version": "Google Open Buildings v3"
16  }
17 }
```

Listing 1: Esquema unificado de documento en mongodb

## 4.3. Indexación espacial

Se implementaron índices `2dsphere` sobre el campo `geometria_edificio` en ambas colecciones de edificios, junto con índices auxiliares para acelerar consultas por fuente y municipio:

```

1 # índice geoespacial 2dsphere (requerido para $geoWithin, $geoIntersects)
2 col.create_index(
3   [("geometria_edificio", GEOSPHERE)],
4   name="idx_geo_edificio"
5 )
6 # índices auxiliares para agregaciones
7 col.create_index("fuente", name="idx_fuente")
8 col.create_index("codigo_mgn", name="idx_codigo_mgn")
```

Listing 2: Creación de índices espaciales en MongoDB

## 4.4. Proceso de cruce espacial y filtro PDET

Un paso crítico en el pipeline de carga fue el cruce espacial entre cada edificio y los polígonos PDET. Solo se insertaron en MongoDB los edificios cuyo centroide cayera *dentro* de algún municipio PDET. Este filtro se implementó de dos formas según la fuente:

- **Google:** uso de `STRtree` de Shapely para consulta espacial eficiente con `tree.query()` y validación con `contains()`.
- **Microsoft:** uso de `geopandas.sjoin()` vectorizado con predicado `within`, procesando todos los edificios de un cuadrante simultáneamente.

## 4.5. Eficiencia de carga

Para garantizar una carga eficiente de grandes volúmenes de datos, se implementó escritura por lotes (*bulk write*) con un tamaño de lote de 2,000 documentos:

```

1 batch.append(InsertOne(doc))
2     count += 1
3
4     if len(batch) >= BATCH_SIZE:
5         col.bulk_write(batch, ordered=False)
6         batch = []
7
8     except Exception as e:
9         log.debug(f" Feature omitida por error: {e}")
10        omitidos += 1
11        continue
12
13    if batch:
14        col.bulk_write(batch, ordered=False)
15
16    log.info(f"  -> {count:,} edificios PDET insertados | {omitidos:,}
17        omitidos (fuera de PDET o invalidos)")
18    total_global += count
19    omitidos_global += omitidos

```

Listing 3: Inserción por lotes con `bulk_write`

El parámetro `ordered=False` permite que MongoDB procese las inserciones en paralelo, mejorando el rendimiento y tolerando errores individuales sin detener el proceso completo.

## 5. Auditoría Inicial de Datos (EDA)

### 5.1. Resumen general

Tras completar la carga de ambos datasets, se realizó un análisis exploratorio de datos (EDA) directamente sobre las colecciones de MongoDB mediante pipelines de agregación. Los resultados globales se presentan a continuación:

Cuadro 4: Resumen general de edificios cargados por fuente

| Métrica                                      | Google      | Microsoft |
|----------------------------------------------|-------------|-----------|
| Total de edificios                           | 2,617,289   | 736,207   |
| Municipios PDET cubiertos                    | 159 / 164   | 82 / 164  |
| Área total estimada (m <sup>2</sup> )        | 216,842,560 | N/D       |
| Área promedio por edificio (m <sup>2</sup> ) | 82.85       | N/D       |
| Área mínima (m <sup>2</sup> )                | 2.51        | N/D       |
| Área máxima (m <sup>2</sup> )                | 20,916.70   | N/D       |
| Confianza promedio                           | 0.7795      | N/D       |
| Confianza mínima                             | 0.65        | N/D       |
| Confianza máxima                             | 0.98        | N/D       |

N/D: No disponible en el dataset de Microsoft.

**Nota sobre Microsoft:** el dataset de Microsoft no incluye el campo `area_in_meters` en sus GeoJSON de distribución. Los valores de `height` y `confidence` registrados en los archivos descargados para Colombia presentaron valores de -1, indicando que no estaban disponibles para esta región en particular.

## 5.2. Cobertura territorial comparativa

Cuadro 5: Cobertura de municipios PDET por fuente

| Categoría                       | Municipios | %       |
|---------------------------------|------------|---------|
| Total municipios PDET           | 164        | 100 %   |
| Con cobertura en Google         | 159        | 96.95 % |
| Con cobertura en Microsoft      | 82         | 50.00 % |
| En ambas fuentes                | 80         | 48.78 % |
| Solo en Google                  | 79         | 48.17 % |
| Solo en Microsoft               | 2          | 1.22 %  |
| Sin cobertura en ninguna fuente | 3          | 1.83 %  |

La cobertura de Google Open Buildings es significativamente superior, alcanzando el **96.95 %** de los municipios PDET, frente al **50.00 %** de Microsoft. Los 5 municipios sin cobertura en Google corresponden principalmente a zonas de selva densa donde la detección satelital es más difícil. Los 3 municipios sin cobertura en ninguna fuente son: **Medio San Juan** (Chocó), **Nóvita** (Chocó) y **Sipí** (Chocó, con solo 1 edificio en Google).

## 5.3. Top 20 municipios por número de edificios — Google Open Buildings

Cuadro 6: Top 20 municipios PDET por número de edificios (Google)

| # | Municipio            | Edificios | Área Total (m <sup>2</sup> ) | Área Prom. (m <sup>2</sup> ) | Confianza |
|---|----------------------|-----------|------------------------------|------------------------------|-----------|
| 1 | Santa Marta          | 183,652   | 16,049,617                   | 87.39                        | 0.7748    |
| 2 | Valledupar           | 171,425   | 15,392,427                   | 89.79                        | 0.7720    |
| 3 | Buenaventura         | 103,890   | 7,666,563                    | 73.80                        | 0.7803    |
| 4 | Turbo                | 78,927    | 6,047,188                    | 76.62                        | 0.7841    |
| 5 | Florencia            | 58,135    | 7,363,355                    | 126.66                       | 0.7753    |
| 6 | San Andrés de Tumaco | 50,493    | 3,778,356                    | 74.83                        | 0.7792    |

| #  | Municipio              | Edificios | Área Total (m <sup>2</sup> ) | Área Prom. (m <sup>2</sup> ) | Confianza |
|----|------------------------|-----------|------------------------------|------------------------------|-----------|
| 7  | Tierralta              | 43,123    | 3,087,942                    | 71.61                        | 0.7859    |
| 8  | Ciénaga                | 42,068    | 3,976,138                    | 94.52                        | 0.7683    |
| 9  | Cajibío                | 41,281    | 2,578,859                    | 62.47                        | 0.7884    |
| 10 | El Tambo               | 38,900    | 2,452,837                    | 63.05                        | 0.7812    |
| 11 | El Carmen de Bolívar   | 38,386    | 2,552,168                    | 66.49                        | 0.7788    |
| 12 | Apartadó               | 36,337    | 3,606,578                    | 99.25                        | 0.7734    |
| 13 | Tame                   | 35,958    | 3,200,497                    | 89.01                        | 0.7838    |
| 14 | Santander de Quilichao | 35,691    | 3,851,789                    | 107.92                       | 0.7834    |
| 15 | Araucuita              | 35,077    | 2,934,442                    | 83.66                        | 0.7865    |
| 16 | Agustín Codazzi        | 34,733    | 2,714,606                    | 78.16                        | 0.7847    |
| 17 | Tibú                   | 32,843    | 2,651,924                    | 80.75                        | 0.7893    |
| 18 | San José del Guaviare  | 32,695    | 3,166,337                    | 96.84                        | 0.7858    |
| 19 | Necoclí                | 32,183    | 2,344,791                    | 72.86                        | 0.7864    |
| 20 | Morales (Cauca)        | 31,057    | 1,969,077                    | 63.40                        | 0.7866    |

#### 5.4. Top 20 municipios por número de edificios — Microsoft Building Footprints

Cuadro 7: Top 20 municipios PDET por número de edificios (Microsoft)

| #  | Municipio              | Edificios |
|----|------------------------|-----------|
| 1  | El Tambo               | 34,366    |
| 2  | Buenaventura           | 33,104    |
| 3  | Santander de Quilichao | 30,842    |
| 4  | San Andrés de Tumaco   | 29,307    |
| 5  | Florencia              | 26,897    |
| 6  | Cajibío                | 23,771    |
| 7  | San Vicente del Caguán | 23,587    |
| 8  | San José del Guaviare  | 21,446    |
| 9  | Puerto Asís            | 21,234    |
| 10 | Orito                  | 17,840    |
| 11 | Chaparral              | 16,848    |
| 12 | Morales (Cauca)        | 16,396    |
| 13 | Valle del Guamuez      | 14,349    |
| 14 | Cartagena del Chairá   | 14,037    |
| 15 | La Macarena            | 12,895    |
| 16 | Caldono                | 12,546    |
| 17 | Mocoa                  | 12,229    |
| 18 | Buenos Aires           | 11,444    |
| 19 | Puerto Rico (Caquetá)  | 11,430    |
| 20 | Planadas               | 11,274    |

#### 5.5. Análisis comparativo entre fuentes

Para los 80 municipios con cobertura en ambas fuentes, se calculó la diferencia en el número de edificios detectados y el ratio Google/Microsoft:

Cuadro 8: Comparativa de detecciones en municipios con cobertura en ambas fuentes (muestra top 10)

| Municipio              | Google  | Microsoft | Diferencia | Ratio G/M |
|------------------------|---------|-----------|------------|-----------|
| Buenaventura           | 103,890 | 33,104    | +70,786    | 3.14      |
| Florencia              | 58,135  | 26,897    | +31,238    | 2.16      |
| San Andrés de Tumaco   | 50,493  | 29,307    | +21,186    | 1.72      |
| Cajibío                | 41,281  | 23,771    | +17,510    | 1.74      |
| El Tambo               | 38,900  | 34,366    | +4,534     | 1.13      |
| Santander de Quilichao | 35,691  | 30,842    | +4,849     | 1.16      |
| San José del Guaviare  | 32,695  | 21,446    | +11,249    | 1.52      |
| Puerto Asís            | 27,581  | 21,234    | +6,347     | 1.30      |
| San Vicente del Caguán | 21,599  | 23,587    | -1,988     | 0.92      |
| Orito                  | 18,074  | 17,840    | +234       | 1.01      |

### 5.5.1. Hallazgos del análisis comparativo

- En **76 de los 80 municipios** comunes, Google detectó más edificios que Microsoft, con un ratio promedio de **1.5x**.
- En municipios como **Buenaventura** (ratio 3.14x) e **Istmina** (ratio 9.24x), la diferencia es muy pronunciada, posiblemente por mejor cobertura de imágenes satelitales de Google en zonas costeras y del Pacífico.
- En **4 municipios**, Microsoft supera a Google: San Vicente del Caguán, La Macarena, Platanadas y Ataco. Esto podría indicar que los cuadrantes de Microsoft tienen mejor cobertura en ciertas zonas del interior del país.
- Municipios como **Orito** (ratio 1.01x) muestran resultados prácticamente idénticos entre fuentes, lo que sugiere consistencia en la detección para esa zona.

## 5.6. Observaciones sobre calidad de datos

Cuadro 9: Comparación de atributos disponibles por fuente

| Atributo                        | Google    | Microsoft |
|---------------------------------|-----------|-----------|
| Geometría (Polygon)             |           |           |
| Área estimada (m <sup>2</sup> ) |           | ×         |
| Nivel de confianza              |           | ×         |
| Altura del edificio             | ×         | ×         |
| Código de ubicación (Plus Code) |           | ×         |
| Cobertura municipios PDET       | 96.95 %   | 50.00 %   |
| Total edificios en PDET         | 2,617,289 | 736,207   |

\* Los campos height y confidence de Microsoft aparecen como -1 para Colombia.



## 6. Análisis Reproducible de Rooftops y Generación de Resultados

### 6.1. Objetivo del análisis

Con el fin de consolidar una metodología reproducible para el análisis del potencial solar en municipios PDET, se desarrolló el script:

02\_rooftop\_analysis.py

Este componente automatiza el cálculo de:

- Conteo total de edificaciones por municipio.
- Área total estimada de techos.
- Métricas de calidad espacial.
- Exportación geoespacial en formato GeoJSON.
- Generación automática de mapas temáticos.

El análisis se ejecuta directamente sobre las colecciones almacenadas en MongoDB, integrando los datasets de Google Open Buildings y Microsoft Building Footprints previamente cargados en el entorno NoSQL.

### 6.2. Ejecución reproducible

La ejecución del pipeline se diseñó bajo un enfoque reproducible mediante parámetros configurables desde línea de comandos:

```
1 python 02_rooftop_analysis.py \  
2 --mongo mongodb://localhost:27017/ \  
3 --out outputs
```

Listing 4: Ejecución del análisis reproducible

Los parámetros soportados por el script son:

Cuadro 10: Parámetros del análisis reproducible

| Parámetro | Descripción                                    |
|-----------|------------------------------------------------|
| -mongo    | URI de conexión hacia MongoDB                  |
| -db       | Nombre de la base de datos utilizada           |
| -out      | Carpeta destino para exportación de resultados |

### 6.3. Metodología implementada

El pipeline automatizado ejecuta las siguientes etapas metodológicas:

1. Carga de geometrías territoriales PDET desde MongoDB.
2. Carga de edificios Google y Microsoft.
3. Conversión de documentos GeoJSON a GeoDataFrames.
4. Validación geométrica de polígonos.

5. Cálculo de áreas en sistema métrico EPSG:9377.
6. Cruce espacial (`sjoin`) mediante predicado `within`.
7. Agregación de conteos y áreas por municipio.
8. Exportación de tablas CSV y capas GeoJSON.
9. Generación automática de mapas temáticos PNG.

## 6.4. Conversión y validación geoespacial

Para garantizar integridad espacial, cada documento almacenado en MongoDB fue convertido a objetos geométricos mediante Shapely:

```
1 geom = d.get(geom_field) or d.get("geometria")
2 g = shape(geom)
```

Listing 5: Conversión de geometrías GeoJSON

Durante el proceso se registraron métricas de calidad espacial:

- Documentos sin geometría.
- Geometrías inválidas.
- Geometrías correctamente cargadas.
- Edificios asignados espacialmente a municipios.

Estas métricas fueron posteriormente exportadas al archivo:

`quality_report.csv`

## 6.5. Cálculo de áreas de techos

Las áreas fueron calculadas utilizando el sistema de referencia métrico oficial:

EPSG:9377

Este CRS permite obtener superficies precisas en metros cuadrados para análisis de potencial solar.

```
1 gdf_metric = gdf.to_crs(epsg=9377)
2 geom_areas = gdf_metric.geometry.area.round(2)
```

Listing 6: Cálculo de áreas en CRS métrico

Cuando el dataset de Google ya contenía el atributo `area_estimada_m2`, el pipeline reutilizaba dicho valor para evitar reprocesamiento innecesario.

## 6.6. Cruce espacial y asignación municipal

La asignación territorial se realizó mediante operaciones espaciales vectorizadas utilizando GeoPandas:

```
1 gg = gdf_google.sjoin(  
2     gdf_mun[  
3         ["codigo_mgn", "nombre_municipio", "geometry"]  
4     ],  
5     how="left",  
6     predicate="within"  
7 )
```

Listing 7: Asignación espacial mediante sjoin

El uso del predicado `within` garantiza que únicamente los edificios completamente contenidos dentro de municipios PDET sean contabilizados.

## 6.7. Agregación de resultados

Posteriormente se realizaron agregaciones estadísticas para calcular:

- Número total de edificios por municipio.
- Área total acumulada de techos.
- Resultados separados por fuente.
- Totales consolidados Google + Microsoft.

```
1 df.groupby(  
2     ["codigo_mgn_mun", "nombre_municipio_mun"]  
3 ).agg(  
4     n_edificios=("geometry", "count"),  
5     area_total_m2=("area_m2", "sum")  
6 )
```

Listing 8: Agregación por municipio

## 6.8. Exportación de resultados

El pipeline exporta automáticamente múltiples salidas para análisis posteriores:

Cuadro 11: Archivos generados por el pipeline reproducible

| Archivo                         | Contenido                              |
|---------------------------------|----------------------------------------|
| rooftop_by_municipio_fuente.csv | Cuentos y áreas por municipio y fuente |
| rooftop_by_municipio.csv        | Totales agregados por municipio        |
| quality_report.csv              | Métricas de calidad espacial           |
| municipios_rooftop.geojson      | Capa geoespacial enriquecida           |
| mapa_rooftop.png                | Mapa temático de edificaciones PDET    |
| run_metadata.json               | Metadatos metodológicos y versiones    |

## 6.9. Generación automática de mapas

Como parte del análisis reproducible, el script genera automáticamente mapas coropléticos utilizando Matplotlib y GeoPandas.

```
1 gdf_mun2.plot(  
2     column="total_edificios",  
3     ax=ax,  
4     legend=True,  
5     cmap="OrRd"  
6 )
```

Listing 9: Generación automática de mapa temático

La Figura 11 presenta el resultado final generado automáticamente por el pipeline.

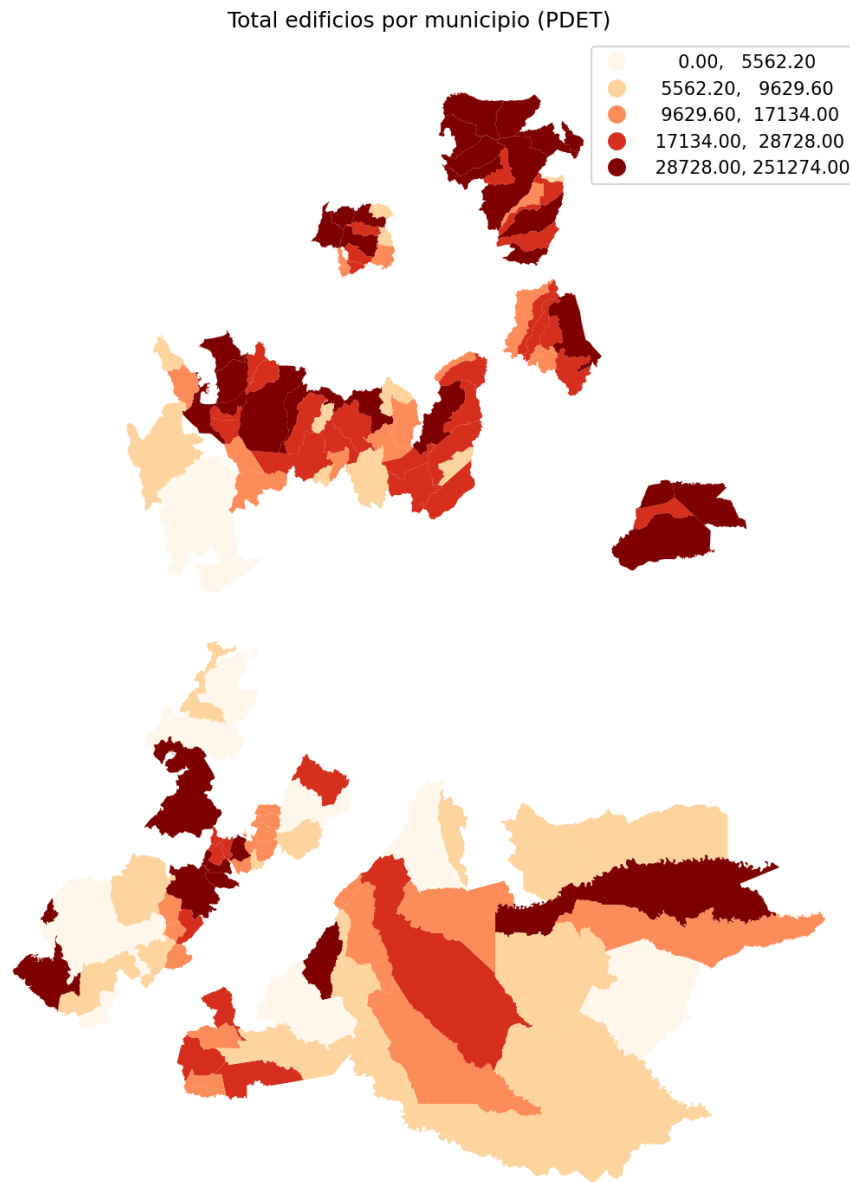


Figura 11: Mapa temático generado automáticamente por el pipeline reproducible. La visualización representa el total de edificaciones detectadas por municipio PDET tras la integración de datasets geoespaciales.

## 6.10. Metadatos y reproducibilidad

Con el objetivo de asegurar trazabilidad y reproducibilidad científica, cada ejecución genera automáticamente un archivo:

`run_metadata.json`

Este archivo almacena:

- Fecha y hora UTC de ejecución.
- Parámetros utilizados.

- Versiones de librerías geoespaciales.
- Número de geometrías procesadas.
- Metodología aplicada.

Un extracto del archivo generado se muestra a continuación:

```

1 {
2   "analysis": "Rooftop Count and Area Estimation",
3   "parameters": {
4     "spatial_join_predicate": "within",
5     "area_crs_epsg": 9377
6   },
7   "inputs": {
8     "municipios_pdet": 164,
9     "google_geometrias_cargadas": 2617289,
10    "microsoft_geometrias_cargadas": 824605
11  }
12 }
```

Listing 10: Metadatos de ejecución reproducible

## 6.11. Resultados generales del análisis

La ejecución del pipeline permitió consolidar información geoespacial reproducible sobre el potencial de techos en municipios PDET.

Los principales resultados obtenidos incluyen:

- Procesamiento de más de 3.4 millones de geometrías de edificios.
- Integración automática entre Google Open Buildings y Microsoft Building Footprints.
- Generación de estadísticas territoriales consolidadas.
- Producción automática de capas geoespaciales compatibles con QGIS y MongoDB.
- Validación espacial reproducible mediante métricas cuantitativas.

Estos resultados constituyen la base para futuros análisis de potencial fotovoltaico y planeación energética territorial dentro del marco PDET.

## 6.12. Hallazgos territoriales del análisis de rooftops

El análisis reproducible permitió identificar diferencias significativas en la distribución territorial de edificaciones dentro de los municipios PDET.

A partir de los resultados agregados obtenidos en:

`rooftop_by_municipio.csv`

se evidenció que los municipios con mayor concentración de edificaciones corresponden principalmente a territorios con mayor densidad urbana y actividad económica regional.

Entre los principales hallazgos obtenidos se destacan:

- **Alta concentración de edificaciones en zonas urbanas consolidadas:** municipios como Santa Marta, Valledupar y Buenaventura presentaron los mayores conteos de edificaciones detectadas dentro de territorios PDET.

- **Distribución desigual del área total de techos:** aunque algunos municipios presentan menos edificaciones, el área promedio de techos es significativamente mayor, indicando presencia de construcciones de gran tamaño o baja densidad urbana.
- **Cobertura territorial heterogénea:** los municipios ubicados en regiones selváticas del Pacífico y Amazonía presentaron menores densidades de detección, lo cual coincide con limitaciones de visibilidad satelital y dispersión poblacional.
- **Mayor cobertura del dataset de Google:** Google Open Buildings logró detectar una cantidad considerablemente superior de edificaciones frente a Microsoft Building Footprints en la mayoría de municipios analizados.
- **Consistencia espacial:** la tasa de asignación territorial obtenida mediante operaciones espaciales *within* confirmó una alta coherencia entre las geometrías de edificios y los límites municipales PDET.

El análisis de áreas acumuladas permitió además estimar de manera preliminar el potencial de superficie disponible para futuras instalaciones fotovoltaicas distribuidas.

Los resultados muestran que municipios con alta concentración de edificaciones también presentan una elevada disponibilidad de área útil de techos, convirtiéndose en candidatos estratégicos para iniciativas de transición energética y generación solar descentralizada.

Adicionalmente, el uso de métricas reproducibles y exportaciones geoespaciales en formatos abiertos facilita futuras etapas de modelamiento energético, simulación fotovoltaica y priorización territorial por parte de entidades gubernamentales como la UPME.